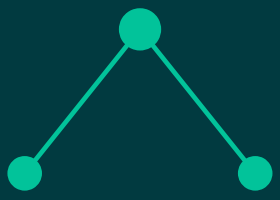
A LEARNING PROJECT



I learned OpenTelemetry by building it.

11 chapters. Real code, real output — every single time.

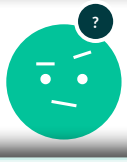
github.com/MaheshPulivarthi18/OpenTelmetry

SWIPE →

THE PROBLEM

2 log files.
0 shared ID.

gateway.log

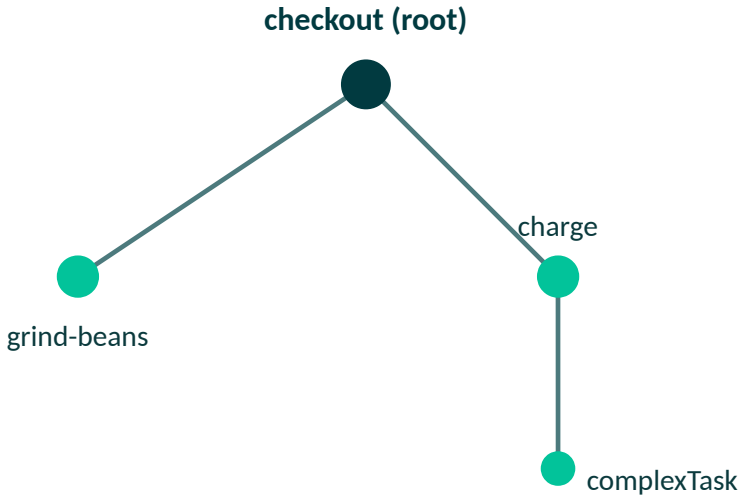


backend.log

Fired 3 requests at once. Couldn't tell which line belonged to which request — from either file.

THE FIX

One ID.
A real tree.



traceId	shared by every span
parentId	who caused whom, automatically
traceparent	survives a real network call

GOING DISTRIBUTED

From one process to a real system

01

Auto-instrumentation

Real spans, zero span code.

02

The Collector

Real OTel Collector, in Docker.

03

Async messaging

PRODUCER/CONSUMER, real time gap.

04

DBs & external APIs

Pool-wait split from query time.

SWIPE →

THE PROOF

The numbers don't lie

2 vs 20

Metrics: a good label kept 20 requests as 2 useful data points.

1 grep

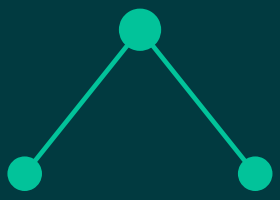
Logs: trace_id injected automatically — one grep, not manual pairing.

2 of 20

Sampling: a 20% head sampler kept 2 traces out of 20.

SWIPE →

WHAT'S NEXT



Fundamentals → Production

- Express, Fastify, NestJS, LoopBack
- gRPC — a genuinely different transport
- Postgres, MySQL, MongoDB, Redis
- RabbitMQ and Kafka — real brokers
- Docker Compose + a real dashboard

Follow along:

github.com/MaheshPulivarthi18/OpenTelmetry

Every chapter: real code, a real run, real output.

THANKS FOR READING